

PROJECT REPORT: SUBSCRIPTION MANAGER AGENT (T4)

Autonomous Financial Assistant & Budget Optimization System



Project Repository:

https://github.com/NagaChaitanya5143/subscription_manager



Group Project Credentials

Field	Student 1 (Team Lead)	Student 2 (Co-Developer)
Full Name	K. Naga Chaitanya	V. Sai Tanmai
Registration Number	12308076	12319651
Roll Number	9	60

1. Executive Summary & Objective

The **Subscription Manager Agent** is an intelligent agent designed to tackle personal finance challenges such as subscription creep, unmonitored recurring renewals, and budget overruns. Built using the **LLaMA 3.1 8B Instruct** model via the **NVIDIA NIM API**, the agent manages recurring services, computes real-time monthly and annual financial metrics, maintains persistent memory across multi-turn interactions, and autonomously recommends optimal cancellation strategies when user budget limits are exceeded.

2. Key Features & Capabilities

- **Dual Tool Integration:** Integrates `add_subscription(name, cost, renewal_date)` to register services with sanitized pricing, and `get_monthly_total()` to query totals.
- **Persistent Memory:** Remembers active subscriptions across multi-turn queries, eliminating stateless arithmetic hallucinations.

- **Yearly Cost View & Renewal Reminders:** Generates annual expenditure projections (12 × monthly cost) along with renewal dates.
- **Autonomous Budget Overage Resolution:** Automatically calculates optimal non-essential subscription cancellation plans to restore budget balance whenever budget thresholds are breached.

3. System Architecture & Tech Stack

```
Language Model : meta/llama-3.1-8b-instruct
API Provider   : NVIDIA NIM API (https://integrate.api.nvidia.com/v1)
Runtime        : Python 3.11 with OpenAI API client & HTTPX truststore
Version Control: Git & GitHub Repository
```

4. Test Scenarios & Experimental Verification

Scenario 1: Initial Subscriptions Within Budget

Input Prompt: "Add Netflix for \$15 renewing on the 5th, and Spotify for \$10 renewing on the 20th. My monthly budget is \$40."

Execution: Called `add_subscription` for Netflix (\$15) and Spotify (\$10), followed by `get_monthly_total`.

Result: Monthly Total: **\$25.00** | Yearly Total: **\$300.00** (Safely within \$40 budget).

Scenario 2: Budget Overflow & Cancellation Suggestion

Input Prompt: "Add Gym Membership for \$50 renewing on the 1st. My budget is \$50. Check if I'm over budget and suggest what to cancel to get back under."

Execution: Added Gym (\$50). Monthly total reached **\$75.00** (\$25 over budget).

Result: Agent identified Gym Membership (\$50) as over budget and suggested canceling it to bring monthly spending back to \$25.00.

Scenario 3: Multi-Turn Memory & Complex Optimization

Input Prompt: "Also add Disney+ for \$14 renewing on the 12th. Show my total, yearly cost, renewal reminders, and best cancellation plan for my \$50 budget."

Execution: Added Disney+ (\$14). Total monthly spend reached **\$89.00** (Yearly: **\$1,068.00**).

Result: Agent formulated the optimal cancellation strategy: Cancel Gym Membership (\$50) and Netflix (\$15) to reduce total spending to **\$24.00/month**, safely under the \$50 budget.

5. How to Run the Application

Navigate to the project workspace and run either of the execution modes:

```
# 1. Run Automated Test Suite
python main.py

# 2. Run Interactive Live Chat Mode
python interactive.py
```